

Smart India Harvest
AgriDSS: An Integrated Agricultural Decision Support System

A Project Report submitted in partial fulfillment of the
requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING

by

Name	Roll No.
Harish Saini	_____
Navya Bhumbak	_____
Anisha Gupta	_____
Bhavya Mittal	_____

under the guidance of

Project Guide

School of Computer Science
University of Petroleum and Energy Studies
Bidholi, Via Prem Nagar, Dehradun, Uttarakhand

May 2025

Candidate's Declaration

We hereby certify that the project work entitled “**Smart India Harvest: An Integrated Agricultural Decision Support System**” submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** at the School of Computer Science, University of Petroleum and Energy Studies, Dehradun, is an authentic record of our work carried out during the period from **January 2025 to May 2025**.

The work presented in this report was completed by our team under the guidance of the project supervisor. The system, documentation, experiments, and implementation described here have not been submitted by us for the award of any other degree or diploma at this or any other university.

Harish Saini

Roll No. _____

Navya Bhumbak

Roll No. _____

Anisha Gupta

Roll No. _____

Bhavya Mittal

Roll No. _____

Date: _____

Place: Dehradun

Certificate

This is to certify that the project report entitled “**Smart India Harvest: An Integrated Agricultural Decision Support System**” submitted by **Harish Saini, Navya Bhumbak, Anisha Gupta, and Bhavya Mittal** is a bonafide record of the project work carried out under my supervision. The report presents the design, implementation, and evaluation of a working mobile and backend platform for agricultural decision support.

The work is found suitable for submission in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**.

Date: _____

Project Guide

Acknowledgement

We would like to express our sincere gratitude to our project guide for the constant support, feedback, and encouragement provided throughout the development of Smart India Harvest. The guidance helped us convert an idea into a working decision support system with a clear technical direction and a practical agricultural purpose.

We are thankful to the School of Computer Science, University of Petroleum and Energy Studies, for providing the environment, facilities, and academic support needed to complete this work. We also thank our course coordinators and faculty members for their suggestions during reviews and demonstrations.

We acknowledge the open-source tools and developer communities behind FastAPI, React Native, Expo, scikit-learn, PyTorch, Hugging Face Transformers, SQLAlchemy, Open-Meteo, and the Groq SDK. These technologies gave us the foundation to build a usable full-stack system instead of a collection of isolated prototypes.

We also thank the UPES CSI Smart India Hackathon 2025 team and reviewers. The project was recognized among the Top 10, and that experience helped us sharpen the product vision, farmer workflow, and implementation priorities.

Finally, we thank our families and friends for their support and patience while we worked through design, model training, backend debugging, mobile testing, and report preparation.

Harish Saini Navya Bhumbak Anisha Gupta Bhavya Mittal

Abstract

Indian farmers, especially small and marginal farmers, make high-impact decisions under uncertainty. A single season can require choices about crop selection, pest control, irrigation, yield expectations, market timing, and government support. Most digital agriculture tools solve only one of these problems at a time. Many of them also assume stable internet connectivity, English fluency, and the ability to interpret raw model outputs. These assumptions do not match the daily reality of many rural users.

Smart India Harvest is an implemented agricultural decision support system that brings multiple advisory functions into one mobile platform. The system combines crop recommendation, plant disease detection, yield prediction, ET_0 -based irrigation guidance, mandi price support, government scheme information, offline knowledge access, and a 7-day farm action plan. The mobile application is built using React Native and Expo, while the backend uses FastAPI, Python, SQLAlchemy, PostgreSQL, scikit-learn, PyTorch, Hugging Face Transformers, Open-Meteo, and Groq-hosted language and speech models.

The core technical idea is formalized as the **HARVEST** framework, which stands for Hybrid Agri-intelligence with Real-time Verification, ET_0 Scheduling, and Transformer-based Diagnostics. It uses a shared context pipeline to collect GPS location, soil profile, crop details, season, weather, and user language before running the relevant models. The crop recommendation model is a tuned Random Forest classifier trained on 2,200 samples across 22 crops, with 99.3% test accuracy and 0.993 macro F1 score. The yield training pipeline uses 27,303 district-level records, a $\log_{1p}$ target transformation, and a Random Forest Regressor with $R^2 = 0.9410$. Disease detection uses a Vision Transformer model for 38 crop leaf classes, and irrigation uses the Hargreaves equation with crop coefficient based water requirement estimation.

The application supports English, Hindi, and Punjabi interfaces, voice commands through Whisper Large V3 Turbo, text-to-speech responses, TTL-based caching,

offline disease image queuing, and an embedded trilingual knowledge base. The result is not just a machine learning demo, but an end to end farmer-facing system that can keep working even when connectivity is weak.

Keywords: Precision Agriculture, Crop Recommendation, Plant Disease Detection, Yield Prediction, Smart Irrigation, Vision Transformer, Random Forest, SHAP, Offline-First Mobile App, Multilingual AI, Agricultural Decision Support System.

Contents

Candidate's Declaration	1
Certificate	2
Acknowledgement	3
Abstract	4
List of Abbreviations	9
1 Introduction	10
1.1 Background	10
1.2 Problem Statement	10
1.3 Project Objectives	11
1.4 Scope of the Project	12
1.5 Major Contributions	12
1.6 Report Organization	13
2 System Analysis	14
2.1 Existing System Review	14
2.2 Gaps Identified	15
2.3 Stakeholders	15
2.4 Functional Requirements	16
2.5 Non-Functional Requirements	18
2.6 Feasibility Study	18
2.6.1 Technical Feasibility	18
2.6.2 Operational Feasibility	19
2.6.3 Economic Feasibility	19
2.7 Comparative Positioning	19
3 Proposed System Design	20
3.1 Overview of the Proposed System	20
3.2 High-Level Architecture	20
3.3 Main Modules	21

3.3.1	Authentication and Onboarding	21
3.3.2	Home Dashboard	21
3.3.3	Crop Recommendation	21
3.3.4	Disease Detection	21
3.3.5	Yield Prediction	22
3.3.6	Smart Irrigation	22
3.3.7	Mandi Prices	22
3.3.8	7-Day Action Plan	22
3.3.9	AI Chat and Voice Assistant	22
3.4	Use Case Diagram	23
3.5	Data Flow	23
3.6	Database Design	23
3.7	API Design	25
3.8	Offline-First State Design	26
4	Algorithms and Mathematical Models	27
4.1	The HARVEST Framework	27
4.2	Crop Recommendation Model	28
4.3	Yield Prediction Model	29
4.4	SHAP Interpretability	31
4.5	Plant Disease Detection	31
4.6	Irrigation Model	32
4.7	Voice Intent Model	32
4.8	Action Plan Rules	33
5	Implementation	34
5.1	Technology Stack	34
5.2	Project Structure	34
5.3	Backend Implementation	36
5.3.1	Context Utilities	36
5.3.2	Soil Lookup	36
5.4	Mobile App Implementation	37
5.4.1	Services	37
5.4.2	Translation and Voice Context	38
5.5	Disease Detection Sequence	38
5.6	Prompting and Structured Output	38
5.7	Offline Implementation	39
5.8	Error Handling and Fallbacks	39
6	Testing, Output Screens, and Results	41
6.1	Testing Strategy	41
6.2	Model Results	41
6.3	Latency and User Experience Results	42
6.4	Output Screen Coverage	43
6.5	Representative User Workflow	44
6.6	Evaluation Against Objectives	46
7	Deployment, Management, and Impact	47
7.1	Deployment Requirements	47
7.1.1	Backend	47

7.1.2	Mobile App	47
7.2	Risk Analysis	48
7.3	Ethical and Social Considerations	48
7.4	Impact of the Project	49
7.5	Maintainability	49
8	Limitations and Future Enhancements	50
8.1	Current Limitations	50
8.2	Future Enhancements	50
9	Conclusion	52
	References	53

List of Figures

3.1	High-level architecture of Smart India Harvest.	20
3.2	Use case diagram showing farmer interactions with the system.	23
3.3	Shared context and advisory data flow.	23
3.4	Connectivity state transition model used by the offline-first layer.	26
4.1	Feature importance of the tuned crop recommendation model.	29
4.2	SHAP feature importance for the yield model.	31
5.1	Disease detection sequence implemented in the app and backend.	38
6.1	Model performance comparison. Crop values show accuracy, yield values show R^2 scaled to percent.	42

List of Tables

2.1	Gaps in Existing Agricultural Advisory Systems	15
2.2	Functional Requirements	16
2.3	Non-Functional Requirements	18
2.4	Comparison with Typical Agricultural Tools	19
3.1	FarmProfile Database Schema	24
3.2	Backend API Endpoints	25
4.1	Crop Recommendation Dataset and Model Details	28
4.2	Crop Recommender Cross-Validation Comparison	29

4.3	Yield Model Comparison on Log-Transformed Target	30
5.1	Technology Stack	34
5.2	Soil Profile Lookup Table	37
6.1	Expected Runtime Behavior by Module	43
6.2	Implemented Mobile Screens	44
6.3	Objective Completion Summary	46
7.1	Project Risks and Mitigation	48

List of Abbreviations

Term	Meaning
AgriDSS	Agricultural Decision Support System
AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
ET_0	Reference Evapotranspiration
ET_c	Crop Evapotranspiration
GPS	Global Positioning System
HARVEST	Hybrid Agri-intelligence with Real-time Verification, ET_0 Scheduling, and Transformer-based Diagnostics
K_c	Crop Coefficient
LLM	Large Language Model
ML	Machine Learning
NPK	Nitrogen, Phosphorus, and Potassium
RF	Random Forest
SHAP	SHapley Additive exPlanations
TTS	Text-to-Speech
ViT	Vision Transformer

Chapter 1

Introduction

1.1 Background

Agriculture remains one of the most important sectors of the Indian economy. It provides livelihood support to a large share of the population and directly affects food security, rural income, water use, and local markets. At the same time, farming has become harder to manage because climate patterns are less predictable, pest pressure changes quickly, and input costs continue to rise.

For a farmer, the difficult part is not only the amount of information available. The difficult part is knowing what information matters today. Soil type, rainfall, current temperature, humidity, crop stage, land size, disease symptoms, past crop choice, and local market conditions all interact. A crop that is suitable in one region may fail in another. A pesticide that works in one situation may be wasteful or unsafe if rain is expected. Irrigation may be needed urgently on one day and unnecessary the next day because of rainfall.

Modern artificial intelligence can help with this decision load, but only if it is packaged in a way farmers can actually use. A model that gives an English output without explanation is not enough. A model that needs perfect network access is not enough. A disease classifier that gives only a class label is not enough. The value of an agricultural decision support system comes from joining model prediction, local context, explanation, timing, and accessibility.

1.2 Problem Statement

Smallholder farmers frequently face the following problems:

- **Fragmented tools:** Weather, disease diagnosis, crop recommendation, yield esti-

mation, and market information are usually available through different systems.

- **Language barriers:** Many advisory systems assume that users can read English technical output.
- **Connectivity issues:** Cloud-only applications become unreliable in rural areas with weak or intermittent mobile internet.
- **Late disease response:** Crop disease identification often depends on manual inspection or delayed expert visits.
- **Water wastage:** Irrigation decisions are often based on habit instead of weather-driven crop water demand.
- **Low trust in model output:** Farmers need reasons, confidence, and action steps rather than black-box predictions.

The problem addressed in this project is therefore the design and implementation of a usable, multilingual, offline-aware agricultural decision support system that can integrate several advisory modules around a common farmer context.

1.3 Project Objectives

The objectives of Smart India Harvest are:

1. Build a mobile application that farmers can use for crop, disease, yield, weather, irrigation, mandi, scheme, and action plan support.
2. Implement a FastAPI backend that exposes reliable agricultural advisory endpoints.
3. Train and integrate machine learning models for crop recommendation and yield prediction.
4. Integrate a Vision Transformer based disease detection model for crop leaf images.
5. Use live weather and GPS context to reduce manual data entry.

6. Provide language support for English, Hindi, and Punjabi.
7. Add voice input and text-to-speech output for hands-free field use.
8. Implement an offline-first layer using cache, queue, and embedded knowledge base patterns.
9. Present model outputs with clear confidence, source attribution, and farmer-friendly explanations.

1.4 Scope of the Project

The project focuses on a working prototype and academic demonstration of an integrated farmer advisory system. It covers mobile app screens, backend API services, database integration, ML inference, weather integration, LLM-generated advisory text, multilingual interface support, and offline behavior.

The system is not a replacement for certified agronomists, laboratory soil tests, or official government advisories. It is designed as a practical decision aid that can guide farmers toward better day-to-day choices and help them decide when expert support is needed.

1.5 Major Contributions

The main contributions of this project are:

- A working React Native mobile app with 18 routes covering major farm workflows.
- A FastAPI backend with endpoints for onboarding, disease detection, voice commands, crop recommendation, mandi prices, yield prediction, irrigation dashboard, action planning, and translation support.
- A tuned Random Forest crop recommendation pipeline with 99.3% test accuracy across 22 crop classes.

- A district-level yield modeling pipeline using 27,303 rows, log1p target transformation, and tree-based model comparison.
- ET_0 based smart irrigation using live Open-Meteo weather and crop coefficient rules.
- Offline-first architecture with AsyncStorage caching, offline disease image queueing, and a trilingual knowledge base.
- A multilingual voice interface using Groq Whisper for transcription and LLaMA 3.3 70B for intent routing.

1.6 Report Organization

Chapter 2 reviews the existing system landscape, user requirements, and feasibility. Chapter 3 describes the proposed system design. Chapter 4 explains the algorithms and mathematical models. Chapter 5 presents implementation details. Chapter 6 discusses testing, outputs, and results. Chapter 7 covers deployment, project management, and impact. Chapter 8 presents limitations and future enhancements. The final chapter concludes the report.

Chapter 2

System Analysis

2.1 Existing System Review

The existing agricultural technology landscape has many useful tools, but most of them solve narrow problems. Some applications provide weather forecasts. Some provide crop prices. Some research prototypes classify plant diseases from images. Some models recommend crops based on soil and climate parameters. These pieces are valuable, but farmers do not experience farm decisions in separate boxes.

For example, crop selection is connected to soil type, current weather, expected rainfall, previous crop, local price, and season. Disease management is connected to image diagnosis, current humidity, rain probability, spray safety, and treatment availability. Irrigation is connected to temperature, crop type, growth stage, rainfall, soil water retention, and land area. A system that ignores these links can give advice that is technically correct but practically incomplete.

2.2 Gaps Identified

Table 2.1: Gaps in Existing Agricultural Advisory Systems

Gap	Typical Situation	Project Response
Integration gap	Separate tools for weather, disease, irrigation, and crop choice	Unified mobile app with shared context and linked backend modules
Language gap	English-first interfaces and technical outputs	Interface and responses in English, Hindi, and Punjabi
Connectivity gap	Cloud-only behavior with no fallback	Cache, offline queue, and embedded knowledge base
Trust gap	Raw predictions without explanation	Confidence scores, reasoning, source attribution, and advisory text
Data entry burden	Farmer manually enters many parameters	GPS, soil lookup, season logic, and live weather auto-fetching
Action gap	Advice is informative but not scheduled	7-day action plan with timing, priority, and weather safety checks

2.3 Stakeholders

The main stakeholders are:

- **Farmers:** Primary users who need practical decisions on crops, disease, irrigation, and market timing.
- **Agricultural advisors:** Experts who may use the system as a first-level diagnostic and planning aid.

- **Government and extension workers:** Users who can share scheme information and promote data-driven advisories.
- **Developers and maintainers:** Team members responsible for backend reliability, mobile updates, model refreshes, and API configuration.
- **Academic evaluators:** Reviewers assessing the project design, implementation, model use, and social relevance.

2.4 Functional Requirements

Table 2.2: Functional Requirements

ID	Requirement
FR1	The user shall be able to select the application language from English, Hindi, and Punjabi.
FR2	The user shall be able to sign up, log in, and persist a local session.
FR3	The user shall be able to complete farm onboarding with land size, soil type, previous crop, and farm status.
FR4	The system shall fetch or infer weather, state, soil values, season, and crop context wherever possible.
FR5	The system shall recommend crops using a trained ML model and enrich results with reasoning.
FR6	The system shall detect crop disease from a leaf image and return top predictions with confidence scores.
FR7	The system shall estimate yield and provide advisory tips for improving production.
FR8	The system shall generate an irrigation dashboard using ET_0 , crop coefficients, rainfall, and land area.

FR9	The system shall show mandi price guidance for regional crops.
FR10	The system shall generate a 7-day action plan with tasks, timing, urgency, and weather-aware restrictions.
FR11	The system shall provide an AI chat assistant for farming queries.
FR12	The system shall accept voice commands and navigate to the relevant screen.
FR13	The system shall cache data and provide offline knowledge when the network is unavailable.
FR14	The system shall queue disease detection requests when the user captures an image offline.

2.5 Non-Functional Requirements

Table 2.3: Non-Functional Requirements

Quality	Requirement
Usability	The app should use simple screens, clear actions, understandable labels, and farmer-friendly responses.
Reliability	Backend endpoints should return structured responses and fallback outputs when external services fail.
Performance	Tabular ML inference should be fast enough for interactive mobile use. Image and LLM requests should provide loading states.
Accessibility	The app should support regional language use, voice input, and speech output.
Maintainability	Services should be separated into backend endpoints, mobile services, screens, and context providers.
Scalability	New crops, languages, schemes, and disease classes should be addable without redesigning the whole system.
Security	API keys and database credentials should be stored in environment variables rather than hardcoded in the report or committed files.

2.6 Feasibility Study

2.6.1 Technical Feasibility

The project is technically feasible because the chosen technologies are mature and compatible. React Native and Expo support cross-platform mobile development. FastAPI provides a clean REST API layer. scikit-learn and PyTorch support the required ML

models. Open-Meteo provides weather data without an API key. Groq provides low-latency LLM and speech endpoints. PostgreSQL through Neon supports persistent farm profile storage.

2.6.2 Operational Feasibility

The application follows natural farmer workflows: check weather, capture leaf image, view recommendations, plan irrigation, ask a question, and review action tasks. The system reduces manual input by auto-fetching context and using stored farm profile data. This makes the system more realistic for repeated use.

2.6.3 Economic Feasibility

The project uses mostly open-source frameworks and free or low-cost APIs. The main recurring costs are server hosting, database hosting, and LLM or speech usage. For a student prototype and hackathon-scale deployment, the cost is manageable.

2.7 Comparative Positioning

Table 2.4: Comparison with Typical Agricultural Tools

Capability	Smart India Harvest	Weather App	Disease App	Market App	Research Model
Crop recommendation	✓	No	No	No	Sometimes
Disease detection	✓	No	✓	No	Sometimes
Yield prediction	✓	No	No	No	Sometimes
Smart irrigation	✓	No	No	No	Sometimes
Mandi support	✓	No	No	✓	No
7-day action plan	✓	No	No	No	No
Multilingual UI	✓	Sometimes	Sometimes	Sometimes	No
Voice navigation	✓	No	No	No	No
Offline cache and queue	✓	Sometimes	Rare	Rare	No

Chapter 3

Proposed System Design

3.1 Overview of the Proposed System

Smart India Harvest is designed as a three-layer system. The mobile application handles user interaction, offline behavior, screen navigation, local cache, voice recording, and result display. The backend handles API routing, database access, ML inference, weather fetching, LLM calls, and advisory generation. External services provide weather, speech transcription, language reasoning, and database hosting.

The system avoids asking the farmer for every technical value. Instead, it builds a context object from location, selected soil type, previous crop, current crop, land area, sowing date, season, weather, and language. This context is then reused across modules.

3.2 High-Level Architecture

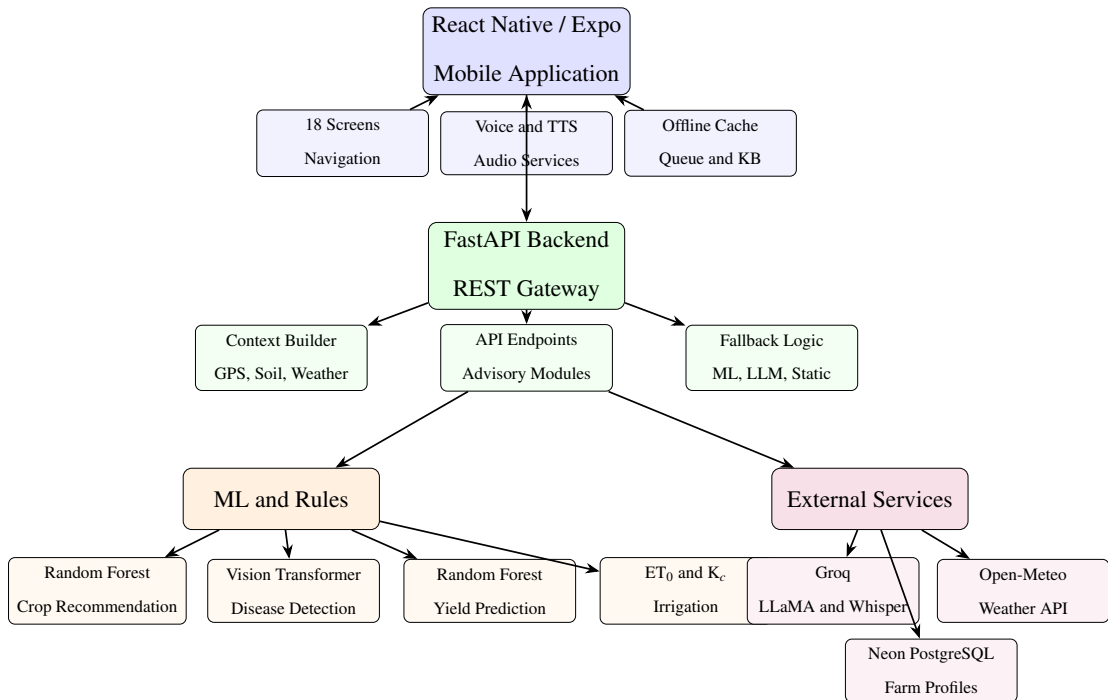


Figure 3.1: High-level architecture of Smart India Harvest.

3.3 Main Modules

3.3.1 Authentication and Onboarding

The mobile application provides language selection, sign up, login, biometric setup, and onboarding. The farm onboarding flow collects land size, land unit, soil type, previous crop, and current farm status. These values are stored and later used by recommendation, irrigation, yield, and action plan modules.

3.3.2 Home Dashboard

The home screen acts as the entry point to major workflows. It shows a farmer greeting, weather summary, active crop information, crop health status, and quick cards for disease detection, crop management, yield prediction, agricultural knowledge, government schemes, mandi prices, and action plans.

3.3.3 Crop Recommendation

The crop recommendation module uses soil type, GPS location, season, previous crop, and live weather. The backend maps the soil type to NPK and pH values, fetches weather from Open-Meteo, predicts the top crops using the Random Forest model, and enriches the output with farmer-friendly reasoning.

3.3.4 Disease Detection

The disease module allows the user to capture or upload a leaf image. If the device is online, the image is sent to the backend endpoint `/api/detect-disease`. The backend runs ViT inference and returns the top five predictions. The mobile app maps the detected label to multilingual disease descriptions and remedies, caches the result, and can read the guidance aloud.

3.3.5 Yield Prediction

The yield module combines crop, state, district, GPS coordinates, soil data, weather, and current year. The trained model estimates district-level production in log scale and converts it back with the exponential inverse. The system also uses the LLM to provide a practical per-hectare estimate and advisory tips because the ML model is trained on macro district production.

3.3.6 Smart Irrigation

The irrigation module estimates reference evapotranspiration using Hargreaves temperature inputs and a latitude-based radiation approximation. It multiplies ET_0 by crop coefficient K_c , subtracts effective rainfall, and converts millimeters of water into liters for the farmer's land area.

3.3.7 Mandi Prices

The mandi module provides regional price guidance for crops relevant to the farmer's location. It uses the selected or detected state and returns price, trend, and market demand style information for the user.

3.3.8 7-Day Action Plan

The action plan module computes crop stage from sowing date, fetches 7-day weather, calculates daily irrigation need, identifies rainy and spray-safe days, and produces practical tasks. When the LLM is available, it generates a structured weekly plan. If it fails, a rule-based fallback still creates irrigation, monitoring, and spray tasks.

3.3.9 AI Chat and Voice Assistant

The AI chat screen works as a farming assistant called Krishi Mitra. The voice assistant records audio through Expo AV, sends it to the backend, transcribes it using Whisper Large V3 Turbo, detects intent using LLaMA 3.3 70B, and returns a screen navigation target with a short response.

3.4 Use Case Diagram

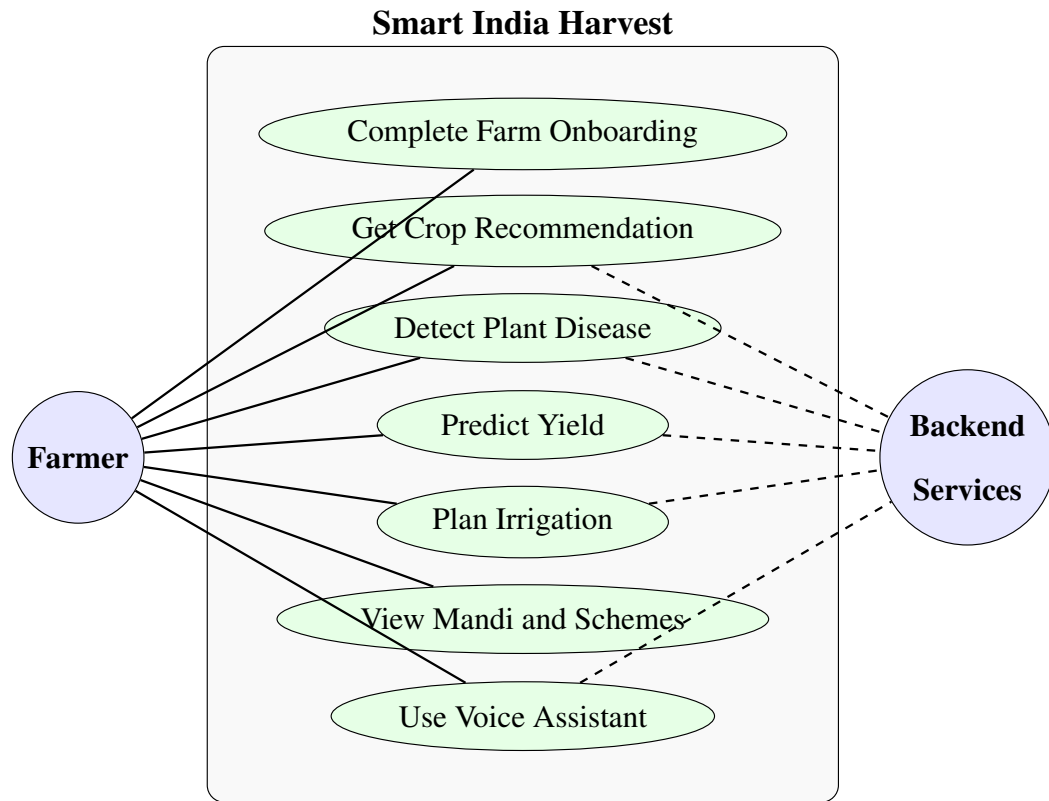


Figure 3.2: Use case diagram showing farmer interactions with the system.

3.5 Data Flow

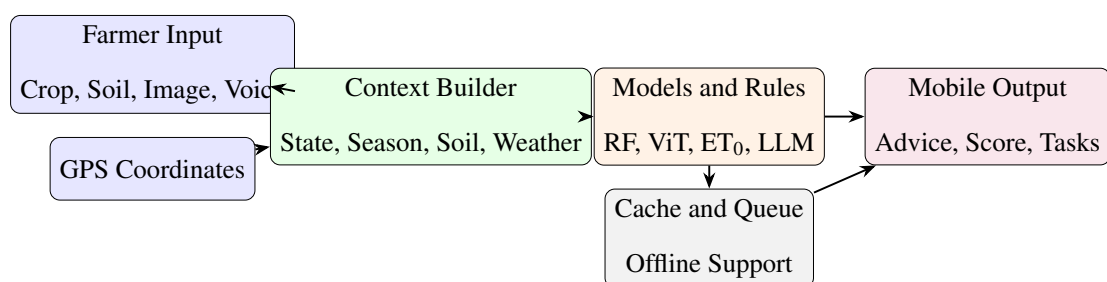


Figure 3.3: Shared context and advisory data flow.

3.6 Database Design

The current persistent database stores farm profiles using SQLAlchemy and PostgreSQL. More entities such as disease history, saved recommendations, and action logs can be added later.

Table 3.1: FarmProfile Database Schema

Field	Type	Purpose
id	Integer	Primary key
user_id	String	User identifier with index
land_size	Float	Farm size value
land_unit	String	Unit such as acre, hectare, or beegha
soil_type	Optional string	Soil category selected in onboarding
previous_crop	Optional string	Previous crop for context
harvest_date	Optional string	Harvest date when available
current_status	String	Empty field or growing status
created_at	Datetime	Profile creation timestamp

3.7 API Design

Table 3.2: Backend API Endpoints

Method	Endpoint	Purpose
GET	/	Root health message
GET	/api/health	Backend and model status
POST	/api/onboarding	Create or update a farm profile
GET	/api/farm-profile/ {user_id}	Retrieve farm profile
POST	/api/detect-disease	Upload leaf image for disease prediction
GET	/api/model-labels	Return disease model labels
GET	/api/translations/ {lang_code}	Return UI translations
POST	/api/voice	Transcribe audio and classify voice intent
POST	/api/ crop-recommendation	Generate ML and LLM crop recommendations
GET	/api/mandi-prices	Generate regional mandi price guidance
POST	/api/yield-prediction	Predict crop yield and advisory tips
GET	/api/irrigation/ dashboard	Return ET ₀ based irrigation dashboard
POST	/api/action-plan	Generate 7-day crop action plan

3.8 Offline-First State Design

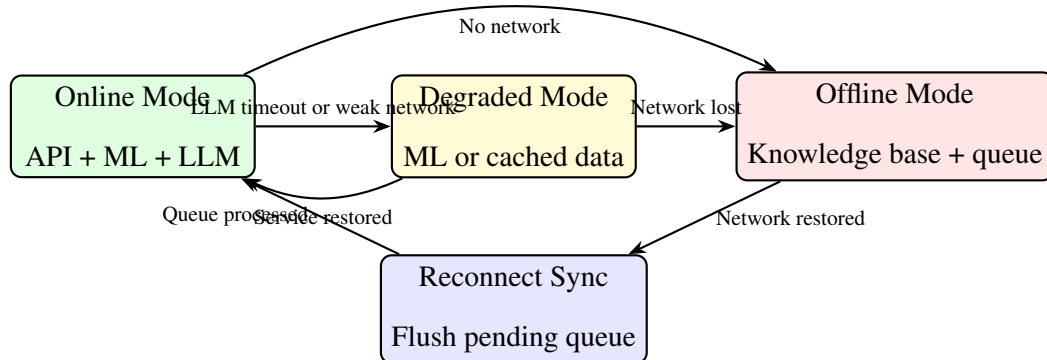


Figure 3.4: Connectivity state transition model used by the offline-first layer.

Chapter 4

Algorithms and Mathematical Models

4.1 The HARVEST Framework

The HARVEST framework is the orchestration pattern used by the system. It is not one model. It is the way the system collects context, runs the relevant model or rule engine, enriches the answer when possible, and still returns usable advice when a service fails.

Algorithm 1 HARVEST Advisory Pipeline

Input: User request u containing crop, soil type, location, language, optional image or audio

Output: Advisory response $\mathcal{R}$ with confidence, source, and action guidance

- 1: Resolve language preference and user profile
 - 2: Fetch GPS based state using bounding box lookup
 - 3: Fetch current and 7-day weather from Open-Meteo
 - 4: Lookup soil NPK and pH from selected soil type
 - 5: Compute current season using calendar logic
 - 6: **if** request type is crop recommendation **then**
 - 7: Run Random Forest crop classifier on $[N, P, K, T, H, pH, R]$
 - 8: Rank top five crops by probability
 - 9: **else if** request type is disease detection **then**
 - 10: Preprocess image and run Vision Transformer inference
 - 11: Return top five disease labels and confidence values
 - 12: **else if** request type is yield prediction **then**
 - 13: Encode state, district, crop, soil, weather, and year features
 - 14: Predict log production and convert using $\exp(\hat{y}) - 1$
 - 15: **else if** request type is irrigation **then**
 - 16: Compute ET_0 , ET_c , effective rainfall, and water requirement
 - 17: **else if** request type is action plan **then**
 - 18: Compute crop stage, irrigation need, rainy days, and spray-safe windows
 - 19: **end if**
 - 20: **if** LLM service is available and the module requires reasoning **then**
 - 21: Generate localized explanation, advisory, or weekly plan
 - 22: Set response source to ML plus AI advisory
 - 23: **else if** ML output is available **then**
 - 24: Format ML or rule output with static explanation
 - 25: Set response source to model or rule fallback
 - 26: **else**
 - 27: Return cached data, offline knowledge, or static safe fallback
 - 28: **end if**
 - 29: **return** $\mathcal{R}$
-

4.2 Crop Recommendation Model

The crop recommendation model predicts suitable crops from a 7-dimensional input vector:

$$\mathbf{x}_{crop} = [N, P, K, T, H, pH, R]^T \quad (4.1)$$

Here N , P , and K are soil nutrient values, T is temperature, H is humidity, pH is soil pH, and R is rainfall. In the deployed system, NPK and pH are derived from the soil profile lookup table, while temperature, humidity, and rainfall come from Open-Meteo.

Table 4.1: Crop Recommendation Dataset and Model Details

Item	Value
Dataset size	2,200 rows
Features	N, P, K, temperature, humidity, pH, rainfall
Target	Crop label
Crop classes	22 crops
Train-test split	80% training and 20% testing with stratification
Model	Tuned Random Forest classifier
Best parameters	250 estimators, max depth 12, sqrt max features, min split 2, min leaf 1
Test accuracy	99.318%
Macro F1	0.99318

Table 4.2: Crop Recommender Cross-Validation Comparison

Model	Mean CV Accuracy	Mean CV Macro F1
Random Forest	0.99375	0.99371
Extra Trees	0.99205	0.99198
SVM with RBF kernel	0.98239	0.98221
Logistic Regression	0.96818	0.96782

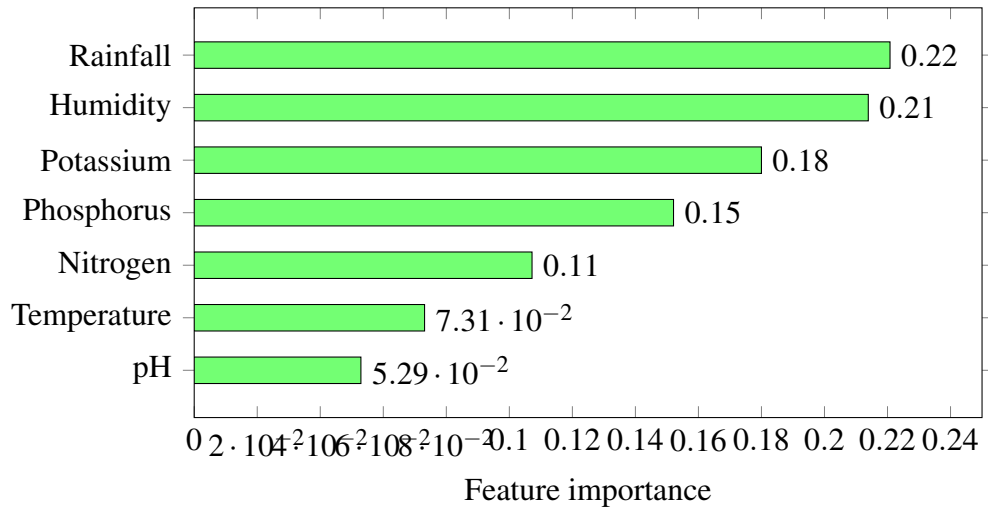


Figure 4.1: Feature importance of the tuned crop recommendation model.

4.3 Yield Prediction Model

The yield training pipeline uses a district-level Indian crop production dataset with 27,303 rows and 21 columns. It covers 29 states, 475 districts, 19 crop groups, and crop years from 2021-22 to 2023-24. The training feature vector is:

$$\mathbf{x}_{yield} = [S_e, D_e, C_e, \phi, \lambda, N, P, K, OC, pH, T, H, R, Sun, Y]^T \quad (4.2)$$

where S_e , D_e , and C_e are encoded state, district, and crop values; ϕ and λ are latitude and longitude; OC is organic carbon; Sun is sunshine hours; and Y is the numeric

year.

The production target was highly right-skewed. The notebook recorded target skewness of 4.28 before transformation and 0.99 after transformation. The model therefore uses:

$$\tilde{y} = \log(1 + y) \quad (4.3)$$

and converts predictions back using:

$$\hat{y} = e^{\hat{\tilde{y}}} - 1 \quad (4.4)$$

The deployed endpoint uses the saved model with live weather, soil lookup, crop encoding, district/state encoding, and current year. Since the ML output represents macro district production, the API also requests an LLM-estimated per-hectare value for user-facing yield guidance. This distinction is important because a farmer expects land-area yield, while the historical dataset is district production data.

Table 4.3: Yield Model Comparison on Log-Transformed Target

Model	R^2	Adjusted R^2	RMSE	MAE
Random Forest baseline	0.9410	0.9409	0.5706	0.2910
XGBoost tuned	0.9357	0.9356	0.5957	0.3707
Random Forest tuned	0.9331	0.9329	0.6079	0.3300
Decision Tree tuned	0.9203	0.9201	0.6635	0.2544
Decision Tree baseline	0.9190	0.9189	0.6686	0.2299
LightGBM tuned	0.9169	0.9167	0.6776	0.4595
Linear Regression	0.0677	0.0657	2.2691	1.9432
SVR fixed	-0.1174	-0.1198	2.4842	1.6225

4.4 SHAP Interpretability

SHAP analysis was run on 500 test samples. The top feature was crop encoding, followed by latitude, soil pH, longitude, and temperature-related features. This is agronomically reasonable because district production strongly depends on crop type and geography.

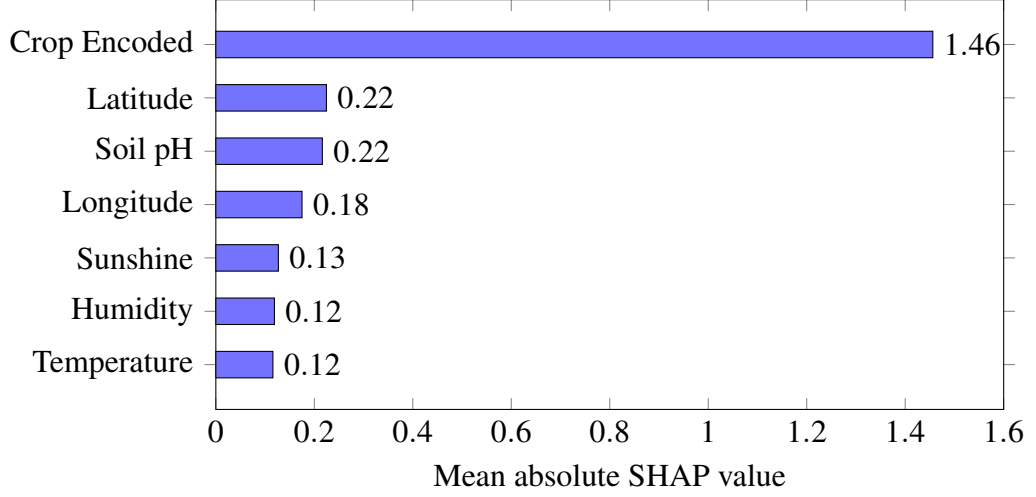


Figure 4.2: SHAP feature importance for the yield model.

4.5 Plant Disease Detection

The disease detection backend loads `wambugu71/crop_leaf_diseases_vit` from Hugging Face. The API accepts an uploaded image, converts it to RGB, preprocesses it using `ViTImageProcessor`, performs inference with `ViTForImageClassification`, applies softmax, and returns the top five predictions.

For an image I , the transformer pipeline can be summarized as:

$$\mathbf{z}_0 = [\mathbf{x}_{cls}; \mathbf{x}_p^1 \mathbf{E}; \dots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{pos} \quad (4.5)$$

$$\hat{y}_{disease} = \text{softmax}(\text{MLP}(\mathbf{z}_L^0)) \quad (4.6)$$

The system returns a ranked prediction list instead of only a single label. This is important in real field conditions where image quality, lighting, and disease similarity

can make the top class uncertain.

4.6 Irrigation Model

The irrigation module uses the Hargreaves equation for ET_0 estimation:

$$ET_0 = 0.0023 \times R_a \times (T_{mean} + 17.8) \times \sqrt{T_{max} - T_{min}} \quad (4.7)$$

The deployed code approximates extraterrestrial radiation R_a from latitude:

$$R_a = 15 + 5 \cos(|lat| \times \pi/180) \quad (4.8)$$

Crop evapotranspiration is:

$$ET_c = K_c \times ET_0 \quad (4.9)$$

Effective rainfall is estimated as 80% of observed rainfall:

$$P_{eff} = 0.8P \quad (4.10)$$

Net irrigation need is:

$$I_{net} = \max(ET_c - P_{eff}, 0) \quad (4.11)$$

For a land area A in hectares, water need in liters is:

$$Water = I_{net} \times A \times 10000/1000 \quad (4.12)$$

4.7 Voice Intent Model

The voice flow has three stages:

1. The mobile app records audio using Expo AV.
2. The backend sends the audio to Groq Whisper Large V3 Turbo for transcription.

3. LLaMA 3.3 70B classifies intent and returns a navigation target such as Weather, DiseaseDetection, Fasalein, YieldPrediction, IrrigationDashboard, MandiPrices, ActionPlan, GovSchemes, AgriKnowledge, or AIChat.

The output is structured JSON containing transcript, intent, navigation target, and response text. This makes the voice assistant usable for screen navigation as well as simple question handling.

4.8 Action Plan Rules

The action plan module first computes crop stage from sowing date. Each crop has approximate durations for seedling, vegetative, flowering, and maturity stages. The system then combines crop stage with 7-day weather, rain probability, wind, humidity, temperature, and ET_0 based irrigation need. It avoids spray tasks on rainy or high-wind days and adds monitoring tasks during high humidity or disease-prone stages.

Chapter 5

Implementation

5.1 Technology Stack

Table 5.1: Technology Stack

Layer	Technologies
Mobile frontend	React Native 0.81, Expo 54, TypeScript, React Navigation, AsyncStorage, NetInfo, Expo Location, Expo Image Picker, Expo AV, Expo Speech
Backend	Python, FastAPI, Uvicorn, Pydantic, SQLAlchemy, httpx, dotenv
Machine learning	scikit-learn, joblib, NumPy, PyTorch, Hugging Face Transformers
Database	PostgreSQL through Neon, SQLAlchemy ORM
External APIs	Open-Meteo, Groq LLaMA 3.3 70B, Groq Whisper Large V3 Turbo
Visualization and report	LaTeX, TikZ, pgfplots, booktabs

5.2 Project Structure

```
smart-india-harvest/  
|-- backend/  
|   |-- main.py  
|   |-- database.py
```

```
| |-- models.py
| |-- crop_recommendation_model.joblib
| |-- RandomForest.pkl
| |-- yield_model/
|     |-- best_model.pkl
|     |-- category_mappings.pkl
|     |-- model_metadata.pkl
|     |-- scaler.pkl
|
|-- crop_rec/
| |-- data/Crop_recommendation.csv
| |-- reports/metrics.json
| |-- reports/feature_importance.csv
| |-- src/train_crop_model.py
|
|-- crop prediction/
| |-- India_Districts_Crop_Production_Processed.csv
| |-- saved_model/
| |-- images/
|
|-- mobile-app/
| |-- App.tsx
| |-- src/navigation/AppNavigator.tsx
| |-- src/screens/
| |-- src/services/
| |-- src/context/
| |-- src/components/
|
|-- report.tex
```

| -- README.md

5.3 Backend Implementation

The backend is implemented in `backend/main.py`. It uses FastAPI with CORS enabled for mobile app access. On startup, it creates database tables and loads ML models. The disease model is loaded through Hugging Face Transformers. The crop recommendation model is loaded from a joblib artifact containing the trained pipeline, feature names, and crop labels. The yield model and category mappings are loaded from the `yield_model` directory.

5.3.1 Context Utilities

The backend contains helper functions for:

- Fetching current and daily weather from Open-Meteo.
- Mapping latitude and longitude to Indian states using bounding boxes.
- Determining the current crop season.
- Calling Groq LLM endpoints.
- Parsing JSON responses from LLM output.
- Computing ET_0 .
- Computing crop stage from sowing date.

5.3.2 Soil Lookup

Eight soil profiles are stored in the backend. Each profile contains reference NPK and pH values:

Table 5.2: Soil Profile Lookup Table

Soil Type	N	P	K	pH
Alluvial	80	40	45	7.0
Black	55	25	60	7.8
Red	40	20	30	6.0
Laterite	35	15	25	5.5
Sandy	25	10	15	6.5
Clay	70	35	55	7.5
Loamy	75	42	50	6.8
Saline	30	12	20	8.5

5.4 Mobile App Implementation

The mobile application is organized into screens, services, context providers, components, and navigation. The app uses `AppNavigator.tsx` to define routes such as `LanguageSelection`, `SignUp`, `Login`, `BiometricSetup`, `Home`, `Weather`, `AIChat`, `DiseaseDetection`, `Onboarding`, `Fasalein`, `SuccessStories`, `Profile`, `AgriKnowledge`, `GovSchemes`, `IrrigationDashboard`, `YieldPrediction`, `MandiPrices`, and `ActionPlan`.

5.4.1 Services

The mobile service layer keeps network and device logic separate from screens:

- `diseaseService.ts` uploads images and handles offline queuing.
- `voiceService.ts` records audio, sends voice commands, and speaks text.
- `weatherService.ts` handles weather requests.
- `translationService.ts` fetches and caches UI translations.

- `offlineStorage.ts` provides TTL-based cache helpers.
- `offlineQueue.ts` queues disease detection images and processes them after re-connection.
- `offlineKnowledgeBase.ts` provides trilingual farming guidance without internet.
- `networkService.ts` monitors connectivity using `NetInfo`.

5.4.2 Translation and Voice Context

The translation provider stores the selected language and exposes a `t()` function for screens. The voice context provides global voice assistant state, making the voice button available across the app.

5.5 Disease Detection Sequence

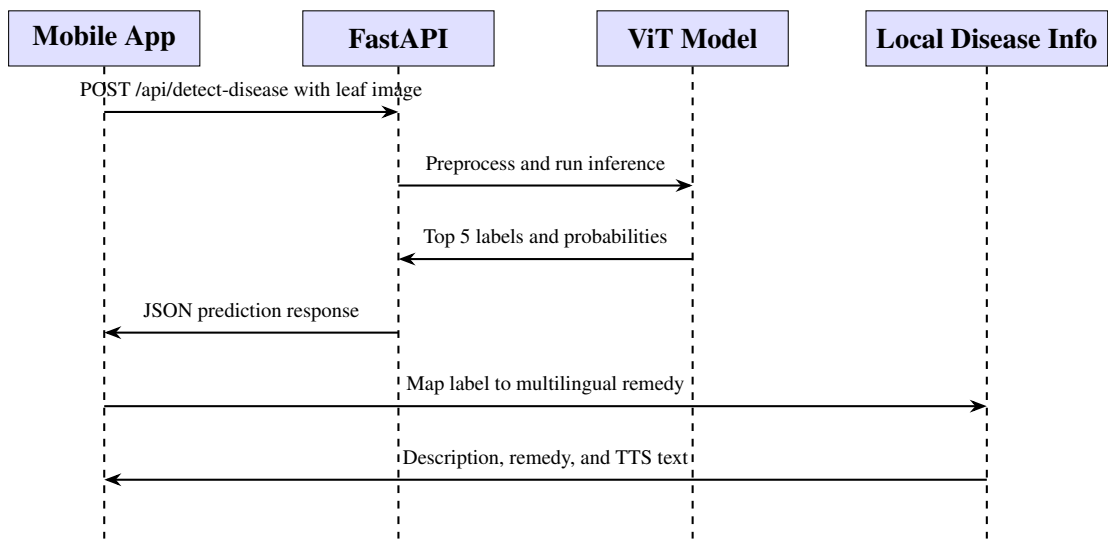


Figure 5.1: Disease detection sequence implemented in the app and backend.

5.6 Prompting and Structured Output

LLM calls are designed to return JSON whenever the response is consumed by the app. For example, crop recommendation enrichment returns a list of objects containing crop, reason, market demand, estimated price, and season match. Action plans return weekly

summary and daily task lists. Voice commands return intent, navigation target, and response text.

The backend includes a parser that first attempts direct JSON parsing and then extracts a JSON object or array if the LLM includes extra text. This is a practical reliability layer for production-style LLM use.

5.7 Offline Implementation

The offline layer is one of the strongest implementation features of the project. It contains:

- **TTL cache:** API responses can be saved with expiry and reused later.
- **Disease image queue:** If the farmer captures a leaf image offline, the app stores the image and queues it for later processing.
- **Reconnect sync:** When connectivity returns, queued items are processed automatically.
- **Knowledge base:** Common crop, pest, soil, irrigation, weather, and government scheme answers are stored locally in English, Hindi, and Punjabi.
- **Offline banner:** The UI can inform the user when the app is operating without network connectivity.

5.8 Error Handling and Fallbacks

The system uses layered fallbacks:

1. Try the best available path, usually ML plus LLM reasoning.
2. If LLM fails, return ML output with static explanation.
3. If ML is unavailable, use LLM-only or rule-based output where appropriate.
4. If the network is unavailable, use cache, queue, or offline knowledge.

This keeps the user experience from collapsing into a blank error screen.

Chapter 6

Testing, Output Screens, and Results

6.1 Testing Strategy

The project was tested at four levels:

- **Model testing:** Evaluate crop recommendation and yield models using held-out data and saved metrics.
- **API testing:** Validate endpoint request and response shapes for farm profile, disease, crop recommendation, yield, irrigation, voice, and action plan flows.
- **Mobile workflow testing:** Navigate through onboarding, dashboard, disease detection, crop management, weather, irrigation, yield, mandi, action plan, and profile screens.
- **Offline testing:** Disable connectivity, verify local knowledge access, cache reads, offline banner behavior, and disease image queueing.

6.2 Model Results

The crop recommendation model achieved strong classification performance. The yield model comparison showed that Random Forest baseline performed best on R^2 and RMSE for the log-transformed production target.

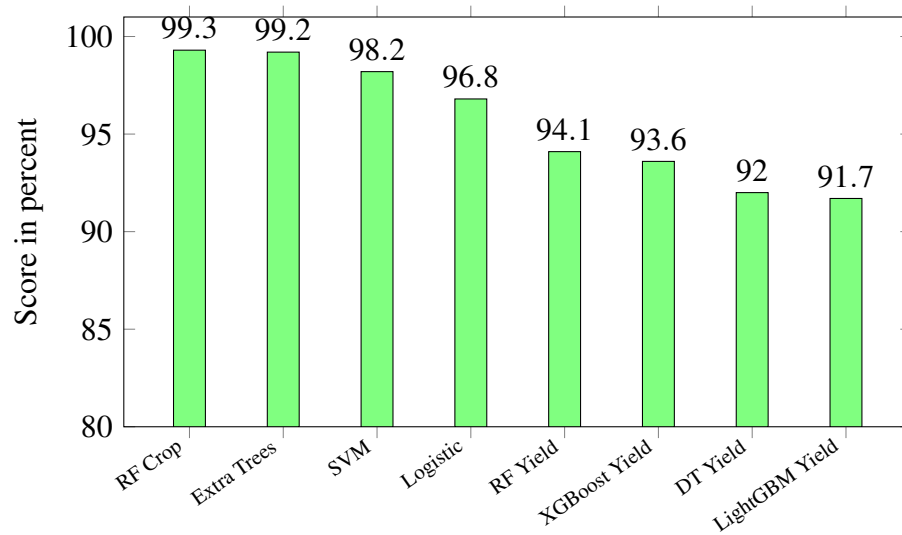


Figure 6.1: Model performance comparison. Crop values show accuracy, yield values show R^2 scaled to percent.

6.3 Latency and User Experience Results

Exact latency varies by device, network, model warm-up, and Groq response time. Based on local implementation behavior and expected endpoint paths, the following latency categories are realistic for the prototype:

Table 6.1: Expected Runtime Behavior by Module

Module	Runtime Type	User Experience
Crop recommendation	Fast tabular ML plus optional LLM	Interactive, shows ranked crops and reasoning
Disease detection	Image upload plus ViT inference	Requires loading state, returns top labels and confidence
Yield prediction	Tabular ML plus LLM per-hectare estimate	Interactive, depends on weather and LLM availability
Irrigation dashboard	Computation plus optional LLM text	Fast, mostly deterministic rules and weather data
Voice assistant	Audio upload, ASR, intent LLM	Requires recording and processing states
Action plan	Rules plus optional LLM planning	Medium latency, returns structured 7-day plan

6.4 Output Screen Coverage

The current mobile application includes the following major screens:

Table 6.2: Implemented Mobile Screens

Screen	Purpose
LanguageSelection	Select English, Hindi, or Punjabi
SignUp and Login	User account and session entry
BiometricSetup	Fingerprint or face authentication setup
Onboarding	Collect land, soil, previous crop, and current farm status
Home	Dashboard and quick navigation
Weather	Current, hourly, and 7-day weather details
DiseaseDetection	Camera or gallery image disease diagnosis
Fasalein	Crop management and crop recommendations
AiChat	Conversational Krishi Mitra assistant
YieldPrediction	Yield estimate and improvement tips
IrrigationDashboard	Soil moisture estimate, water need, schedule, and savings
MandiPrices	Regional commodity price guidance
ActionPlan	7-day crop task plan
GovSchemes	Central and state scheme information
AgriKnowledge	Offline and online farming knowledge
SuccessStories	Farmer story feed and submission
Profile	Farm profile, settings, and language access

6.5 Representative User Workflow

A typical user journey is:

1. The farmer selects Hindi or Punjabi during first launch.
2. The farmer creates an account and completes onboarding with land size and soil type.

3. The home screen shows the current location and weather summary.
4. The farmer opens crop management and receives top crop suggestions generated from soil and live weather.
5. The farmer captures a leaf image in disease detection and receives a confidence-ranked disease result.
6. The farmer checks irrigation, where the app recommends water volume and best irrigation time.
7. The farmer opens the action plan and receives daily tasks for the next week.
8. If the network fails, cached data and offline knowledge remain available.

6.6 Evaluation Against Objectives

Table 6.3: Objective Completion Summary

Objective	Status	Evidence
Mobile farmer app	Completed	React Native app with 18 routes
FastAPI backend	Completed	Implemented endpoints in <code>main.py</code>
Crop recommendation ML	Completed	99.3% test accuracy
Yield prediction ML	Completed	$R^2 = 0.9410$ training result
Disease detection	Completed	ViT model integration
Smart irrigation	Completed	ET_0 and K_c computation
Voice assistant	Completed	Whisper and LLM intent endpoint
Offline support	Completed	Cache, queue, and knowledge base
Multilingual support	Completed	English, Hindi, and Punjabi

Chapter 7

Deployment, Management, and Impact

7.1 Deployment Requirements

7.1.1 Backend

The backend requires Python, dependencies from `requirements.txt`, a PostgreSQL connection string, and a Groq API key. Environment variables should be stored in a local `.env` file:

```
DATABASE_URL=postgresql://user:password@host/database
GROQ_API_KEY=your_groq_api_key
```

The backend can be started with:

```
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

7.1.2 Mobile App

The mobile app requires Node.js, Expo, and dependencies from `package.json`. The backend IP address is configured in `src/config.ts`. The Expo development server can be started with:

```
npx expo start --clear
```


7.2 Risk Analysis

Table 7.1: Project Risks and Mitigation

Risk	Effect	Mitigation
Weak internet	LLM, weather, and image up-load may fail	Cache, offline queue, fallback rules
LLM malformed JSON	App may not parse response	JSON extraction parser and static fallback
Model mismatch	Prediction may be less useful for unseen crops or districts	Confidence display, future re-training, local validation
Incorrect soil lookup	NPK values may not match actual field	Encourage soil test integration in future version
Image quality issues	Disease prediction may be uncertain	Return top five predictions and confidence
API key exposure	Security and billing risk	Use environment variables and avoid hardcoding secrets

7.3 Ethical and Social Considerations

The system must be presented as an advisory tool, not a guaranteed decision maker. Farmers should not be encouraged to apply chemicals or change crop plans without considering local expert advice, input availability, safety instructions, and government recommendations. Disease remedies should be treated as first-level guidance and validated for local conditions.

The app can improve access to agricultural information for users who are not comfortable with English. However, multilingual support must be tested carefully because literal translation can change meaning in agricultural contexts. The system also needs privacy care because farm profiles, locations, and user activity can be sensitive.

7.4 Impact of the Project

Smart India Harvest has practical value because it joins several everyday farming decisions into one accessible workflow. Its impact can be seen in four areas:

- **Decision quality:** Crop, irrigation, disease, and yield decisions use data rather than only habit.
- **Accessibility:** Language and voice support make the app easier for more users.
- **Continuity:** Offline cache and queue features keep the app useful when connectivity is poor.
- **Explainability:** Confidence scores and reasoning help users understand why advice is shown.

7.5 Maintainability

The codebase is structured in a way that supports future improvements. Backend logic is centralized in API endpoints and helper functions. Mobile device features are separated into services. Screens are separated by workflow. Offline behavior is handled by dedicated storage, queue, network, and knowledge base services. This makes it easier to add new crops, disease classes, languages, or advisory modules.

Chapter 8

Limitations and Future Enhancements

8.1 Current Limitations

- **Static soil values:** Soil NPK and pH are inferred from soil type lookup. The system does not yet read live soil sensor data or lab reports.
- **District production vs farm yield:** The yield ML model is trained on macro district production. The app adds LLM-based per-hectare estimation, but a direct farm-level yield dataset would be better.
- **Vision model domain shift:** Plant disease datasets often contain clean leaf images. Real farm images may have clutter, poor lighting, mixed symptoms, or multiple diseases.
- **LLM dependency:** High-quality natural language reasoning and voice intent classification require an external API key and network access.
- **Limited language set:** The app currently focuses on English, Hindi, and Punjabi. India needs more regional languages.
- **Market data realism:** Mandi price guidance needs integration with official or live commodity price APIs for production deployment.
- **No full security layer yet:** The current prototype uses simple local session behavior and should be extended with robust authentication for production.

8.2 Future Enhancements

1. **Soil test integration:** Allow farmers to enter lab report values or scan soil health cards.

2. **IoT support:** Add soil moisture, temperature, humidity, and NPK sensors for live field telemetry.
3. **On-device ML:** Convert selected models to TensorFlow Lite or ONNX Runtime for offline inference.
4. **Expanded disease coverage:** Retrain the disease model with field images from Indian crops and local disease classes.
5. **Official market APIs:** Integrate e-NAM or state mandi feeds for real price data.
6. **More languages:** Add Marathi, Bengali, Tamil, Telugu, Gujarati, Kannada, and Odia.
7. **Expert review mode:** Allow KVK or agricultural advisors to review and approve recommendations.
8. **Notification system:** Send alerts for rain, irrigation windows, spray warnings, and harvest reminders.
9. **Farm history analytics:** Track crop cycles, disease history, irrigation use, and production outcomes.
10. **Better authentication:** Add secure backend-based login, role management, and encrypted user data.

Chapter 9

Conclusion

Smart India Harvest successfully demonstrates a complete agricultural decision support system that is broader than a single machine learning model. It combines mobile usability, backend APIs, crop recommendation, disease detection, yield prediction, irrigation scheduling, mandi guidance, action planning, multilingual interaction, voice support, and offline-first behavior into one project.

The project shows that agricultural AI becomes more useful when it is tied to real context. Soil type, GPS location, current season, live weather, crop stage, user language, and land area are all used to shape the advice. The crop recommendation model achieved 99.3% test accuracy, and the yield modeling pipeline achieved $R^2 = 0.9410$ on the log-transformed target. The irrigation module provides a practical ET_0 based method for water planning, and the offline layer ensures that the app remains useful even under poor connectivity.

Most importantly, the system is designed around the farmer's workflow. It does not simply show model output. It turns the output into recommendations, explanations, timings, and action steps. With future work on live soil data, official price APIs, expanded languages, and field-level validation, AgriDSS can become a stronger foundation for accessible precision agriculture in India.

References

- [1] R. G. Allen, L. S. Pereira, D. Raes, and M. Smith, *Crop Evapotranspiration: Guidelines for Computing Crop Water Requirements*. FAO Irrigation and Drainage Paper 56, Food and Agriculture Organization, Rome, 1998.
- [2] G. H. Hargreaves and Z. A. Samani, “Reference crop evapotranspiration from temperature,” *Applied Engineering in Agriculture*, vol. 1, no. 2, pp. 96-99, 1985.
- [3] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *International Conference on Learning Representations*, 2021.
- [4] D. P. Hughes and M. Salathe, “An open access repository of images on plant health to enable the development of mobile disease diagnostics,” *arXiv preprint arXiv:1511.08060*, 2015.
- [5] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, pp. 5-32, 2001.
- [6] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785-794, 2016.
- [7] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [8] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [9] S. Ramirez, “FastAPI framework documentation,” <https://fastapi.tiangolo.com/>, accessed 2025.
- [10] Expo, “Expo documentation,” <https://docs.expo.dev/>, accessed 2025.
- [11] Open-Meteo, “Open-Meteo weather forecast API,” <https://open-meteo.com/>, accessed 2025.
- [12] Hugging Face, “Transformers documentation,” <https://huggingface.co/docs/transformers/>, accessed 2025.